Exercise 5

Eric Hannus - 654906
ELEC-E8126 - Robotic manipulation

March 20, 2023

1 Grasp planning approach & Bonus: grasp robustness metric

For planning the grasp my approach is to randomly sample two points on different edges, check whether the grasp is stable and how good its grasp quality is if it is stable, i.e. how robust the grasp is, and continue sampling grasps while storing the best found so far.

To compare the robustness of different grasps and select the best one found within the maximum iterations given to the algorithm, I check how large the angle between the normal vector at the contact point going into the polygon (center of friction cone) and the line connecting the contact points is. If this angle exceeds the friction angle, it means the connecting line goes outside the friction cone and the grasp is unstable, as described in Chapter 12.2.3 of [1]. Thus, the grasp is more robust the smaller this angle is, because then the grasp is further from being unstable. To form the quality metric for robustness, I evaluate this angle at both contact points and select the larger one, as we want the larger (worse) angle to be as small as possible. This is because it is not very helpful if one angle is very small while the other is large, as it is enough for the connecting line to fall outside one of the friction cones for the grasp to be unstable. I then scale this angle to a value between 0 - 1 for stable grasps, and negative values for unstable grasps, so that the robustness metric has an easily comparable value. The best value is 1, and it is achieved when the angle at both contact points is zero, that is, the grasping points are connected by a line going straight through the middle of both friction cones. Thus the robustness metric is calculated as follows:

$$quality_metric = \frac{\alpha_\mu - max(angle_{p1}, angle_{p2})}{\alpha_\mu}$$

Where α_μ is the friction angle, and the angles at each contact point described previously are given by $angle_{p1}$ and $angle_{p2}$. It is worth noting that it is not possible to achieve a grasp with the perfect score 1.0 for all polygons, as this requires that there are parallel edges that can be gripped.

I let my algorithm sample points 1000 times (set in "max_iter" parameter in main() function), and keep track of the best grasp found while by rejecting unstable grasps and keeping track of the best quality value observed so far. When tasked with finding the optimal

grasp (as opposed to finding the best grasp within 1000 random samples as it is currently used for) the algorithm is probabilistically complete, because given enough iterations the algorithm would eventually sample the very best grasp and store it (provided that we could stop the algorithm when it has been found).

The code contains various functions for, e.g., extracting edges from given vertices, sampling points, plotting, and evaluating grasp quality. One notable function is "get_curve_orientation" where I calculate the orientation of the polygon, that is whether the vertexes are given in a clockwise or counterclockwise order, using the method described in `https://en.wikipedia.org/wiki/Curve_orientation`. This information is then used when calculating normal vectors at the grasp points, to select which normal vector is pointing into the polygon and which one is pointing outwards, so that I can put the friction cones and approach vector at the correct sides. In a counter-clockwise polygon the inside of the polygon will be on the left side of an edge, while the inside will be on the right side for a polygon where edges are defined in clockwise order.

2 How to run grasp planning code

My grasp planning is carried out in a Python script called "GraspPlanning.py". To run this script go to the folder "~/ros/src/exercise5" and run "python src/GraspPlanning.py", so that it finds correct text files to read vertexes from and write grasp points to. My implementation only uses the additional libraries Numpy and Matplotlib.

3 Visualization for various polygons

The "GraspPlanning.py" script creates a window with a figure showing the polygon with the most robust grasping points found in the 1000 given iterations, according to the metric defined earlier. The figure shows both grasping points with corresponding friction cones and approach vectors. The legs of the friction cones are plotted at a fixed length of 0.25 units, as only the angles they define are of interest in this case. The line connecting the grasping points is also shown, and the title contains grasping point coordinates and approach angles, as well as the value of the robustness metric for the grasp. The figure also contains consecutive vertexes "A", "B", and "C", where "B" is a point on the convex hull as described in `https://en.wikipedia.org/wiki/Curve_orientation`. These points make it easy to see directly from the plot whether the polygon was defined in a clockwise or counterclockwise order, and they visualize the points that are used to calculate this information in the code. Figure. 1 to Figure. 7 show the results when "GraspPlanning.py" is applied on various polygons, both concave and convex, and oriented both clockwise and counterclockwise.

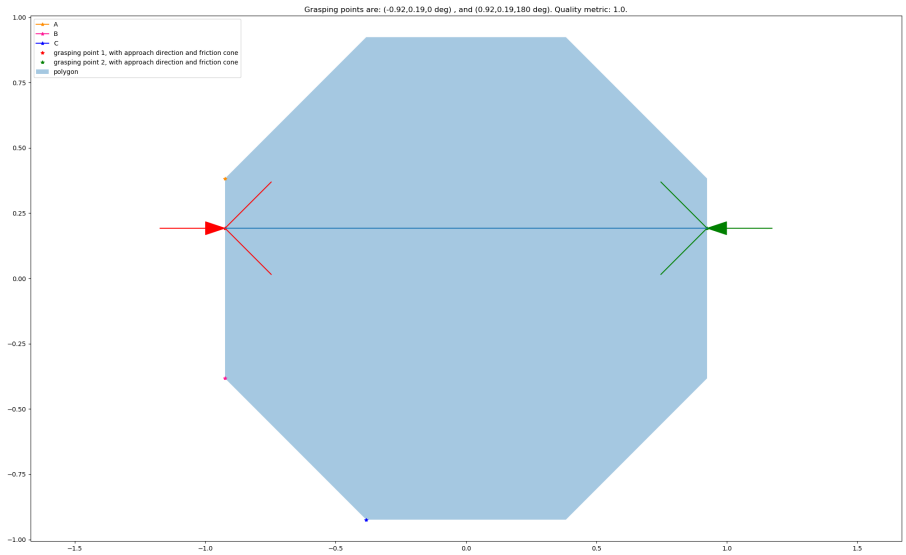

Figure 1: The original octagon.

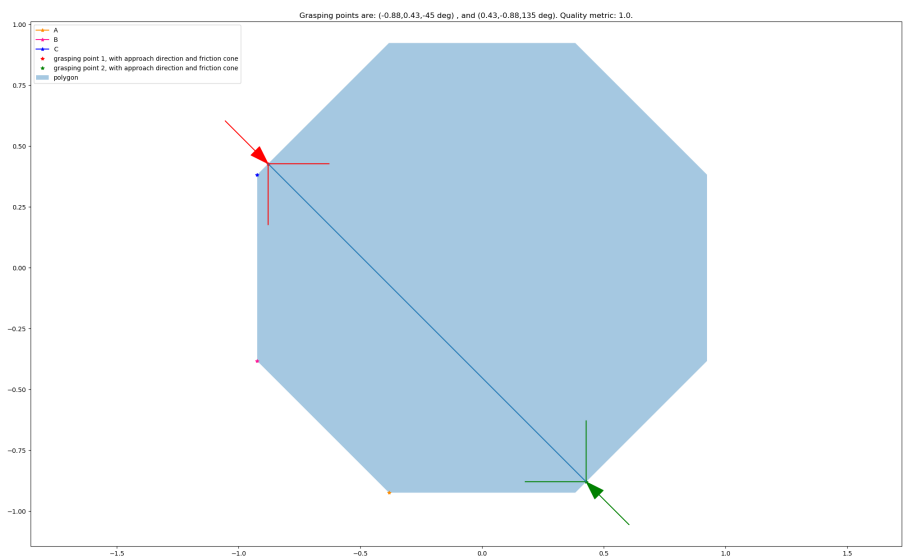

Figure 2: The octagon with vertexes defined in clockwise order.

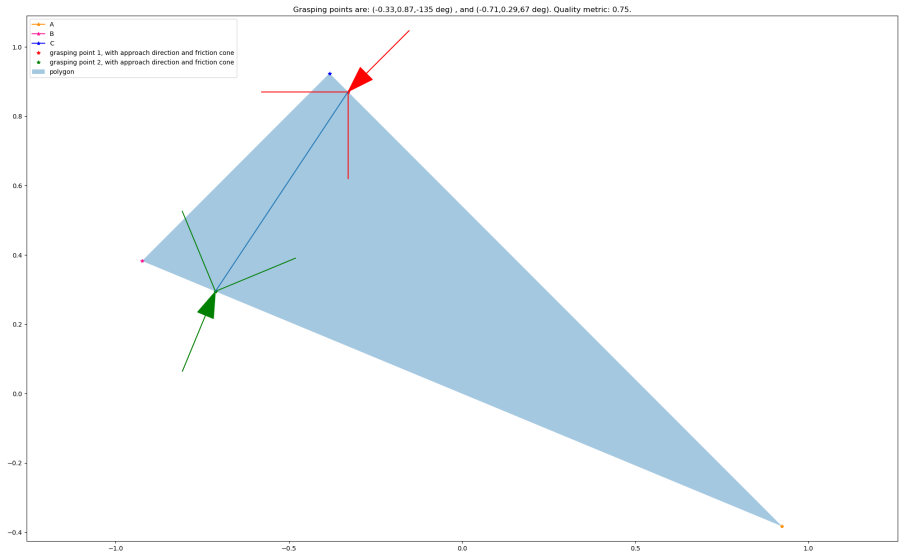

Figure 3: A triangle defined in clockwise order.

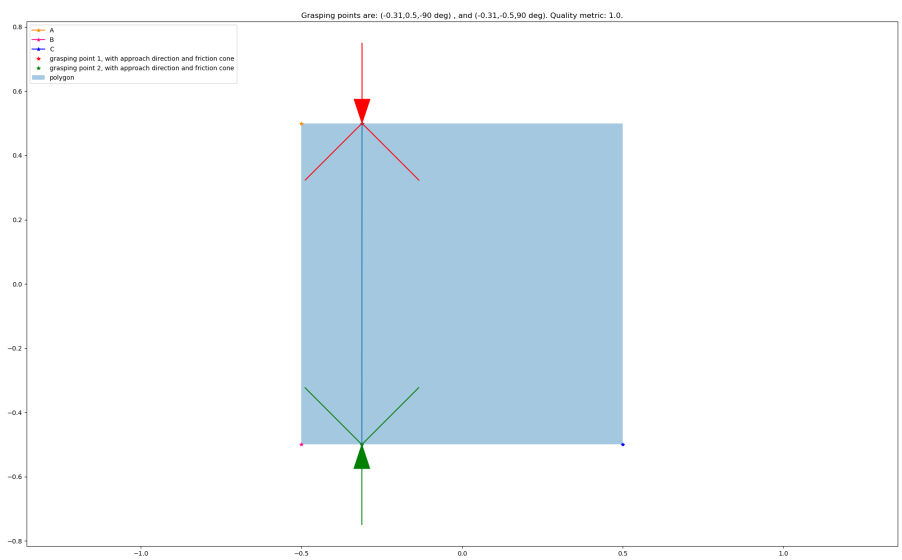

Figure 4: A square defined in counter-clockwise order.

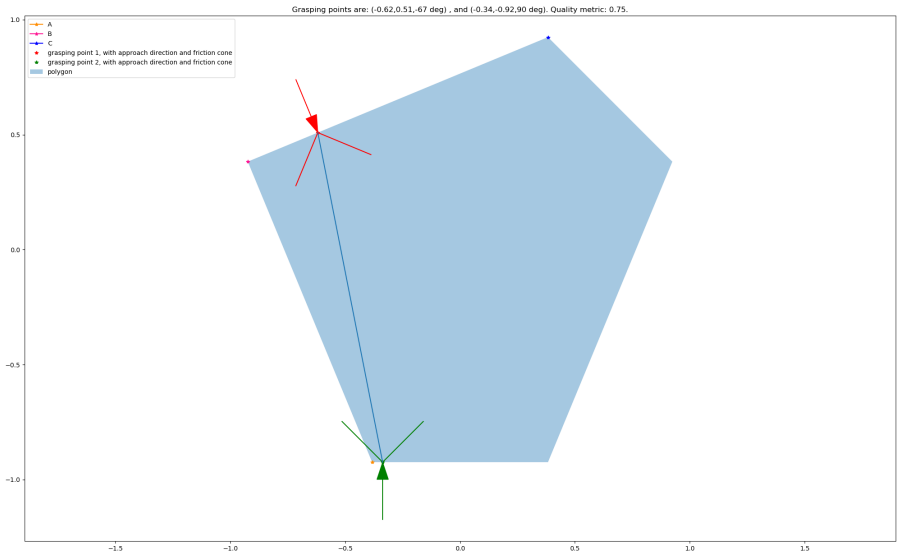

Figure 5: A pentagon defined in clockwise order, that we get from removing vertices from the original octagon shape and changing the order of remaining vertices.

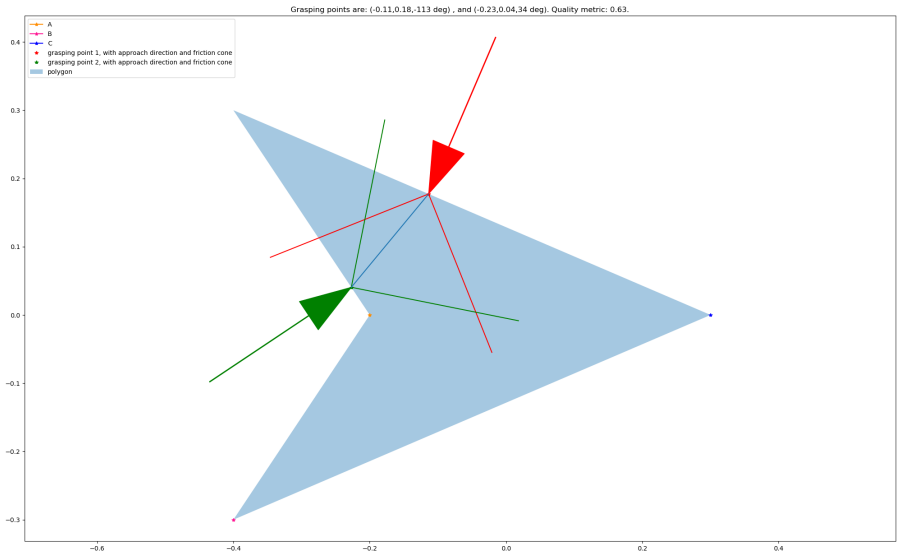

Figure 6: A concave quadrilateral defined in counter-clockwise order.

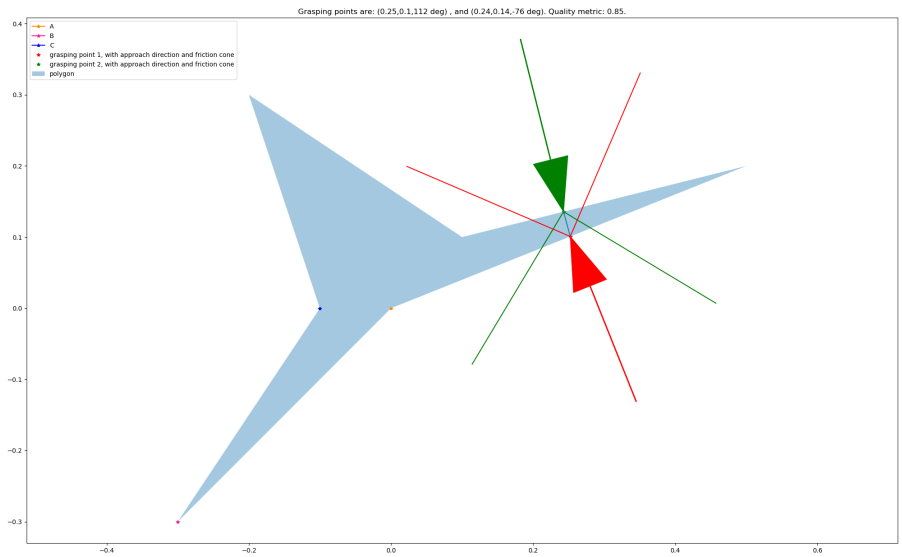

Figure 7: A concave six-edged polygon defined in clockwise order.

4 Discussion

From the figures presented in the previous section we can see that the algorithm succeeds in generating stable grasps for both convex and concave polygons, irrespective of whether they were defined with clockwise or counterclockwise orientation. The code runs quickly for the 1000 sampling iterations and finds grasps that are stable and robust according to the defined criterion, and as mentioned previously the optimal grasps for this criterion would be found given infinite time. It is worth noting that there are planned grasps that are valid as they are stable, but that the ROS simulation cannot execute as the robot arms do not reach so far. For example, when both grasp points are placed close to the midpoint between the robots and they are expected to grasp the object from the positive and negative direction of the y-axis, the arms cannot reach these positions. A stable grasp that ROS fails to execute as the robot arms do not reach to the grasping points is shown in Figure. 8, and a stable grasp that ROS manages to execute is shown in figure Figure. 9 for comparison.

Form closure is discussed in Chapter 12.1.7 and force closure in Chapter 12.2.3 of [1]. For the grasp to be in form closure, the contact points of the robot fingers should constrain the object and prevent any motion, without considering friction forces. According to Theorem 12.6 of [1] at least four point contacts are needed for first-order form closure (curvature of contact not considered) for a planar object. This is obviously not the case for our grasp, as the two robot fingers only provide two point contacts in total, so the grasp is not in first order form closure. It is possible that a higher order form closure could be achieved for some polygons and some grasps, for example if the robot fingers grasp the slimmest part of a roughly hourglass-shaped object so it cannot be moved due to the curvature of the surface

at the grasping points. But in the general case the object is not in form closure.

An object is in force closure if any wrench can be resisted with enough force at the friction contacts. As discussed in Chapter 12.2.3 of [1], and mentioned earlier when my grasp planning approach was presented, a planar object is in force closure if the line connecting the contact points stays within both friction cones. As my grasp planning approach is designed to fulfil this condition, all grasps generated are in force closure, and no additional contacts are needed to achieve it.

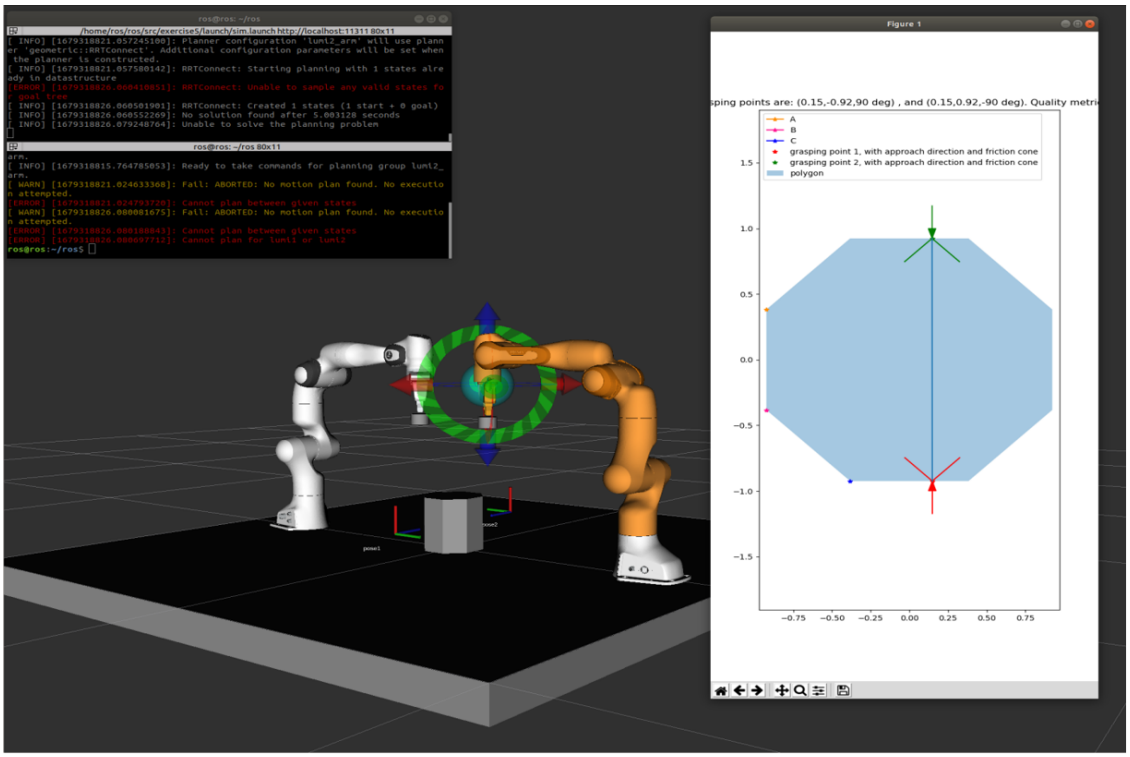

Figure 8: A stable grasp that the robots cannot not reach.

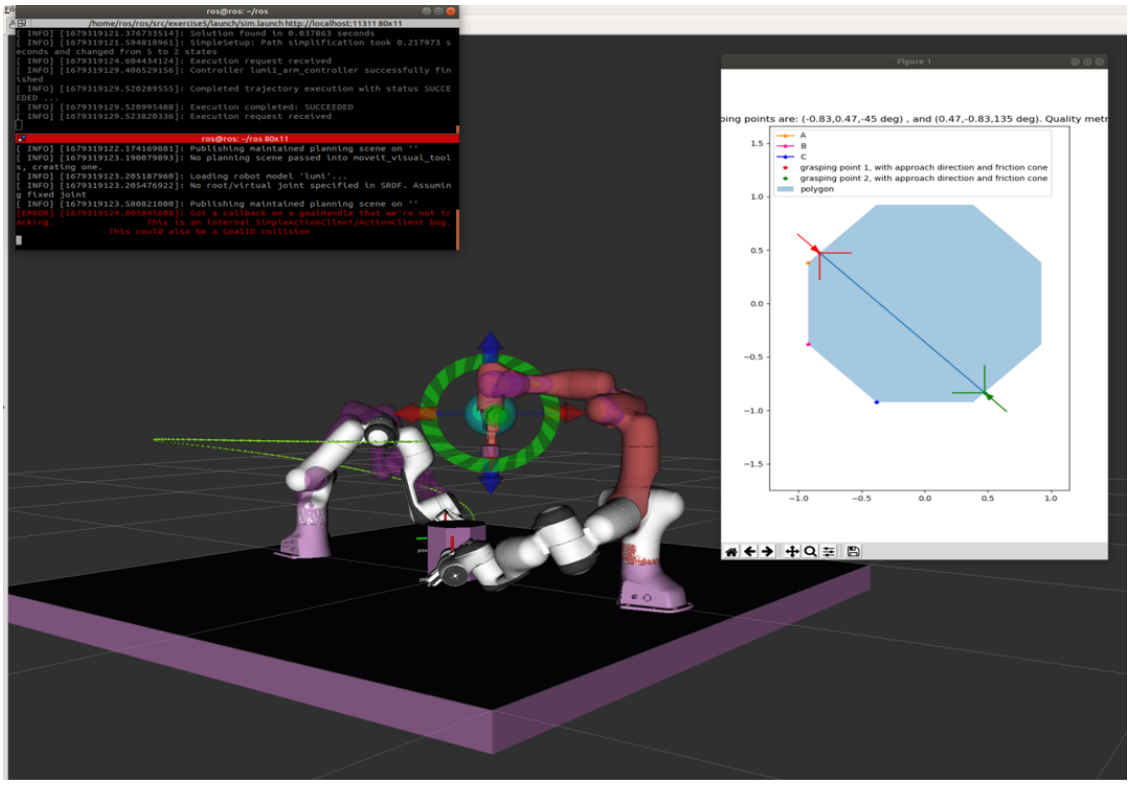

Figure 9: A stable grasp the robots manage to reach.

5 Estimate of time spent

I estimate that I have spent around 20h on this exercise. In the beginning I spent some of the time looking at some stability metrics in literature, like independent contact regions (ICRs), before deciding on this simpler implementation I have used here. Quite a lot of time was also spent testing the implementation on various polygons to make sure different cases like convex and clockwise oriented polygons were also handled correctly.

References

[1] K. M. Lynch and F. C. Park, *Modern Robotics: Mechanics, Planning, and Control*. USA: Cambridge University Press, 1st ed., 2017.